Guía práctica: pendientes y cómo poner todo a funcionar

Proyecto: alta disponibilidad de una base de datos en 3 equipos · Repositorio: `github.com/NohelEstradaP/ha_db_ip`

Esta guía es la versión corta y en orden de lo que falta. La explicación de cada decisión está en `PLAN_ALTA_DISPONIBILIDAD.md`; aquí se indica la sección del plan cuando hace falta más detalle.

---

1. Estado actual

Ya está hecho

- ☑ Plan completo (MD + PDF)
- ☑ Todos los archivos del clúster en el repositorio: Docker Compose, Patroni, HAProxy, esquema SQL
- ☑ Aplicación de prueba: generador de carga, monitor y verificador de integridad
- ☑ Instalador `instalar.sh` para macOS y Pop!_OS
- ☑ Scripts: `iniciar.sh`, `crear_bd.sh`, `estado.sh`, `pctl.sh`, `env_app.sh`, `probar_puertos.sh`
- ☑ Panel web en tiempo real (`./scripts/panel.sh`), probado en el laboratorio con failover y reingreso
- ☑ Laboratorio local (3 nodos en una sola máquina), probado en la Mac:
- ☑ Failover al apagar el primario: unos 22 s
- ☑ Partición de red del primario: unos 27 s, sin dos primarios
- ☑ Reingreso con `pg_rewind` y verificador OK (datos idénticos en los 3)

Falta (lo hace el equipo, en las máquinas reales)

#	Pendiente	Dónde	Sección de esta guía
1	Crear la cuenta de Tailscale	Navegador	§3
2	Correr `instalar.sh` en la Mac	n1	§4
3	Correr `instalar.sh` en las 2 Pop!_OS (**nunca probado en Linux**)	n2, n3	§4
4	Desactivar *key expiry* en Tailscale para los 3	Navegador	§5
5	Verificar que los 3 se ven por red	Los 3	§5
6	Levantar el clúster real	Los 3	§6
7	**Validar la replicación Mac (ARM) ↔ PC (x86)**	Cualquiera	§7
8	Crear la base de datos	Uno	§8
9	Prueba de humo con carga	Los 3	§9
10	Pruebas de falla P1–P9 con evidencias	Los 3	§10
11	Ensayar la demo y preparar la red de respaldo	Los 3	§11
12	Escribir el informe	n/a	§12
13	Borrar el laboratorio de la Mac cuando ya no se use	n1	§14

---

2. Checklist completo

Para marcar a medida que se avanza.

Preparación - ☐ Cuenta de Tailscale creada (una sola para los 3 equipos) - ☐ Decidido qué equipo es cada nodo: Mac = `n1`, PC 1 = `n2`, PC 2 = `n3` - ☐ Los 3 equipos con cargador conectado

Instalación - ☐ `instalar.sh` terminado en n1 (Mac), con las contraseñas guardadas - ☐ `instalar.sh` terminado en n2 con las mismas contraseñas - ☐ `instalar.sh` terminado en n3 con las mismas contraseñas - ☐ En n2 y n3: cerrar sesión y volver a entrar (grupo `docker`) - ☐ Docker Desktop (Mac): "Start when you sign in" activado y Memory ≥ 4 GB - ☐ Tailscale: *key expiry* desactivado en los 3

Clúster - ☐ `tailscale ping` funciona entre los 3 - ☐ `./scripts/iniciar.sh` en los 3 → 1 Leader, 1 Sync Standby, 1 Replica - ☐ `./scripts/probar_puertos.sh` → todo `OK` desde los 3 - ☐ Switchover a la Mac y de regreso, sin errores - ☐ `./scripts/crear_bd.sh` → 100 cuentas, suma 1,000,000 - ☐ Panel HAProxy (`http://<IP>:7000`) con 1 primario y 2 réplicas en verde

Pruebas - ☐ 3 generadores corriendo, monitor ≥ 90 cambios/min (esperado: 540) - ☐ `verificar.py --esperar` → `RESULTADO: OK` - ☐ P1 a P9 hechas y registradas - ☐ Cada equipo (incluida la Mac) apagado al menos una vez - ☐ Evidencias guardadas (capturas, logs, CSV, video)

Demo e informe - ☐ Demo ensayada de principio a fin - ☐ Hotspot o router de respaldo probado - ☐ Informe terminado

---

3. Paso 0: antes de empezar

1. **Cuenta de Tailscale:** entrar a `https://login.tailscale.com` y crear la cuenta (con Google, GitHub o Microsoft). Se usa **la misma cuenta** en los 3 equipos.
2. **Asignar nombres:**

Nodo	Equipo
`n1`	Mac (Apple Silicon)
`n2`	PC Pop!_OS 1
`n3`	PC Pop!_OS 2

1. **Tener a mano** un lugar para guardar las 4 contraseñas que genera el instalador del primer equipo (notas, gestor de contraseñas). No se suben a GitHub.

> Tailscale **no** depende de ninguna de las 3 PC. Es una red en malla coordinada desde internet, así que apagar cualquier equipo no afecta la conexión entre los otros dos.

---

4. Paso 1: instalar cada equipo

Se hace en este orden: **n1 primero** (genera las contraseñas) y luego n2 y n3.

4.1 En cada equipo

```
git clone https://github.com/NohelEstradaP/ha_db_ip.git ~/hadb
cd ~/hadb
./instalar.sh
```

> Si en la Mac ya tienes el proyecto en `~/Documents/alta_disponibilidad_db`, puedes correr `./instalar.sh` desde ahí mismo en lugar de clonar otra vez.

4.2 Qué va a pasar y qué responder

El instalador...	Qué hacer
Pide la contraseña de `sudo`	Escribir la contraseña del usuario del equipo
Mac: abre Docker Desktop	Aceptar los términos la primera vez y esperar
Mac: abre la app de Tailscale	Clic en el ícono de la barra de menú → *Log in* → misma cuenta
Pop!_OS: muestra una URL de Tailscale	Abrirla en el navegador e iniciar sesión con la misma cuenta
"¿Desactivar la suspensión automática?"	**Sí**
"¿Qué nodo es este equipo?"	`n1`, `n2` o `n3` según la tabla del paso 0
"IP de n1 / n2 / n3"	La de este equipo aparece por defecto (Enter). Las de los otros dos se ven en el panel de Tailscale (*Machines*) o con `tailscale ip -4` en cada uno
"¿Es el PRIMER equipo que configuras?"	**n1:** `s`. El instalador muestra 4 líneas `..._PASSWORD=...`: **cópialas**
	n2 y n3: `n` y pegar las 4 contraseñas de n1, una por una
Construye las imágenes	Esperar de 2 a 5 minutos

Al final muestra **"Nodo nX listo"** y los siguientes pasos.

> **Detalle de las IP:** como n1 se instala primero, quizá todavía no conozcas las IP de n2 y n3. Hay dos opciones: - **(a)** Instalar Tailscale primero en los 3 (en Pop!_OS: `curl -fsSL https://tailscale.com/install.sh | sh && sudo tailscale up`) para tener las 3 IP antes de empezar. - **(b)** Escribir cualquier IP válida en n1 y, cuando ya se conozcan todas, corregirlas con `./instalar.sh --solo-env` (al preguntar "¿Es el PRIMER equipo?" responder `n` y pegar las mismas contraseñas).
>
> Las IP deben quedar **iguales en los 3** `.env` antes del paso 3.

4.3 Después del instalador

- **Pop!_OS (n2 y n3):** cerrar sesión y volver a entrar, para poder usar `docker` sin `sudo`. Los scripts funcionan igual sin hacerlo, porque usan `sudo` si hace falta.
- **Mac (n1):** en Docker Desktop → *Settings*:
- *General* → "Start Docker Desktop when you sign in" ✓

- *Resources* → Memory ≥ 4 GB
- **Revisar el `.env`** de cada equipo: `bash cat ~/hadb/.env | grep -v PASSWORD` `N1_IP`, `N2_IP` y `N3_IP` deben ser iguales en los 3. `NODE_NAME` y `NODE_IP` cambian según el equipo.

4.4 Si el instalador falla en Pop!_OS

Es la parte que no se pudo probar antes. Si algo falla: 1. Copiar el mensaje de error completo. 2. Revisar si es de red (por ejemplo `403`, `Could not resolve`): cambiar de red, como pasó con la Mac. 3. Volver a correr `./instalar.sh`, que continúa donde se quedó. 4. Si sigue fallando, hacer ese paso a mano con la sección indicada del plan (Docker: plan §7.2; Tailscale: plan §6.1).

---

5. Paso 2: revisar Tailscale y la red

1. **Desactivar la expiración de llaves** (si no, a los 180 días un equipo sale de la red sin aviso): `https://login.tailscale.com/admin/machines` → en cada equipo ⋯ → **Disable key expiry**.
2. **Probar la conexión** desde cada equipo hacia los otros dos: `bash tailscale ping <IP_del_otro> ping -c 3 <IP_del_otro>` - Sale `via <ip>:<puerto>`: conexión directa. - Sale `via DERP(...)`: pasa por un relay. Funciona igual, con un poco más de latencia.
3. **En la Mac**, el comando `tailscale` de Homebrew **no** habla con la app. Usar el de la app: `bash alias tailscale=/Applications/Tailscale.app/Contents/MacOS/Tailscale tailscale status` Para dejar el alias permanente, agregarlo a `~/.zshrc`.

---

6. Paso 3: levantar el clúster

En **los 3 equipos**, con pocos minutos de diferencia (etcd necesita al menos 2 encendidos para arrancar):

```
cd ~/hadb
./scripts/iniciar.sh
```

El script levanta los contenedores, espera el quórum de etcd y espera a que haya un primario. Al final debe mostrar algo así:

```
| Member | Host        | Role         | State     | TL | Lag in MB |
| n1     | 100.x.x.x   | Replica      | streaming |  1 |         0 |
| n2     | 100.x.x.x   | Leader       | running   |  1 |           |
| n3     | 100.x.x.x   | Sync Standby | streaming |  1 |         0 |
```

Si justo al arrancar no aparece `Sync Standby`, esperar 10 segundos y correr `./scripts/estado.sh`: Patroni lo asigna en su primer ciclo.

Luego, desde **cada** equipo:

```
./scripts/probar_puertos.sh      # todo debe decir OK
```

Y abrir en el navegador `http://<IP_de_cualquier_nodo>:7000`: en el bloque `primario` debe haber 1 en verde y en `replicas` 2 en verde.

Si `iniciar.sh` se queda esperando el quórum de etcd: algún nodo no alcanza a los otros. Revisar que los 3 tengan `iniciar.sh` corriendo, que las IP del `.env` sean correctas y los firewalls (plan §5 y §17).

---

7. Paso 4: validar la Mac como primario y como réplica

Es **el riesgo técnico más importante** y el laboratorio no lo cubre, porque ahí todo corría en la misma arquitectura. Hay que comprobar que PostgreSQL replique bien entre la Mac (ARM) y las PC (x86).

El objetivo es que **un nodo ARM (Mac) y uno x86 (Pop!_OS)** hayan sido primario cada uno, con los demás replicando desde él.

```
./scripts/pctl.sh list                  # ver quién es Leader y quién es Sync Standby
./scripts/pctl.sh switchover --force    # el Sync Standby pasa a primario
./scripts/pctl.sh list                  # esperar a que los 3 estén running/streaming con lag 0
# repetir switchover hasta que haya sido primario un nodo de cada arquitectura
```

> **Importante:** con replicación síncrona, Patroni **solo** acepta como nuevo primario a la réplica que en ese momento es `Sync Standby` (si no, responde `412 candidate name does not match with sync_standby`). Es una protección: así nunca se promueve un nodo al que le falten datos confirmados. Por eso el cambio se hace **sin `--candidate`** (Patroni elige a la síncrona) y se repite hasta que el nodo deseado sea `Leader`.

Pasa si después de cada cambio los 3 nodos terminan en `running` / `streaming` con lag 0.

> ✓ **Validado el 2026-10-23 en las máquinas reales:** el clúster arrancó con n3 (Mac ARM) como primario y n2 (Pop!_OS x86) replicando desde él. Después n3 → n1 → n2, y con n2 (x86) como primario las dos Mac replicaron con los datos intactos (100 cuentas, suma 1,000,000).

Si falla (errores tipo `incompatible` o la Mac no arranca como réplica): aplicar el plan B del **plan §10.6**, que corre el contenedor de la Mac como x86 con Rosetta.

---

8. Paso 5: crear la base de datos

Una sola vez, en cualquier equipo:

```
./scripts/crear_bd.sh
```

Esperado: `cuentas: 100, suma: 1000000.00`. Si se vuelve a correr, no hace nada, porque detecta lo que ya existe.

---

9. Paso 6: carga, monitor y verificación

9.1 En cada equipo, una terminal con el generador

```
cd ~/hadb
source scripts/env_app.sh
python app/generador.py --nombre n1      # n2 / n3 según el equipo
```

9.2 En el equipo que se proyecte, otra terminal con el monitor

```
cd ~/hadb && source scripts/env_app.sh
python app/monitor.py
```

Después de un minuto debe mostrar unas **180 transacciones/min**, **540 cambios/min [OK]** y **Suma 1,000,000.00 [OK]**.

9.3 Panel web en tiempo real (recomendado para la demo)

En uno o dos equipos:

```
cd ~/hadb
./scripts/panel.sh
```

Abrir `http://localhost:8080`, o desde otro equipo o un celular `http://<IP del equipo>:8080`. Si `ufw` está activo en Pop!_OS, permitir el puerto: `sudo ufw allow 8080/tcp`.

Parte del panel	Qué muestra
Encabezado	Sano / Degradado / **Sin primario**
Indicadores	Cambios por minuto (✓ si ≥ 90), primario actual, suma de saldos y **duración del último failover**
Tarjetas de nodo	Rol, estado, timeline, lag, filas, suma, etcd y a quién manda HAProxy las escrituras
Replicación	Dibujo de quién replica a quién (síncrona / asíncrona)
Gráfica	Cambios por minuto de los últimos 10 min, línea del mínimo de 90 y marca ⚡ en cada failover
Eventos	Caídas, cambios de rol, failover con su duración y re-sincronización
Generadores / movimientos	Filas, `seq`, faltantes y en qué nodo se escribió cada movimiento

9.4 Verificación de integridad

```
cd ~/hadb && source scripts/env_app.sh
python app/verificar.py             # con carga: suma y "faltan 0" bien en los 3
# Detener los 3 generadores (Ctrl+C) y luego:
python app/verificar.py --esperar   # → RESULTADO: OK. 3 nodos con datos idénticos...
```

10. Paso 7: pruebas de falla

10.1 Rutina para cada prueba

1. Clúster sano (`./scripts/estado.sh`) y los 3 generadores corriendo al menos 2 minutos.
2. Anotar la hora (`date +%T`) y quién es el primario.
3. Provocar la falla (tabla de abajo).
4. Mirar el panel: muestra solo cuánto duró el failover ("Último failover") y si la suma sigue en 1,000,000.
5. Dejar el nodo caído 2 minutos: los cambios/min deben seguir por encima de 90.
6. Restaurar el nodo y esperar a que vuelva a `streaming` (`./scripts/estado.sh`).
7. Ctrl+C en los generadores y correr `python app/verificar.py --esperar`.
8. Guardar evidencias (§10.3).

10.2 Pruebas

ID	Qué hacer	Cómo restaurar	Esperado
P1	Apagar una **réplica** normalmente (menú → Apagar)	Encenderla	Sin interrupción
P2	Apagar una **réplica** con el botón (mantener presionado)	Encenderla	Sin interrupción
P3	Apagar el **primario** con el botón	Encenderlo	Nuevo primario en 20–30 s
P4	Quitar la red al **primario**: WiFi off, cable fuera o `sudo tailscale down`	Reconectar o `sudo tailscale up`	Nuevo primario en 20–30 s; el aislado se degrada solo
P5	Quitar la red a una **réplica**	Reconectar	Sin interrupción
P6	`sudo reboot` en cualquier nodo	Esperar	Vuelve solo como réplica
P7	Apagar un nodo 30–60 min con carga	Encenderlo	Se pone al día solo
P8	`./scripts/pctl.sh switchover --force`	n/a	Cambio de primario en pocos segundos
P9	En el primario: `docker compose stop patroni`	`docker compose start patroni`	Nuevo primario en 2–8 s

- Después de encender un equipo apagado, **no hay que correr nada**: Docker levanta los contenedores solo. En la Mac, Docker Desktop tiene que estar configurado para iniciar con la sesión (paso 4.3).
- Si un nodo no regresa (`start failed`): `./scripts/pctl.sh reinit hadb <nodo> --force`.

10.3 Evidencias de cada prueba

```
mkdir -p evidencias
./scripts/pctl.sh list    > evidencias/pX_lista_$(date +%H%M).txt
./scripts/pctl.sh history > evidencias/pX_historial.txt
docker compose logs --since 30m patroni > evidencias/pX_patroni_$(hostname).log
python app/verificar.py --esperar > evidencias/pX_verificar.txt
```

Además: capturas del monitor antes, durante y después, y un video de la P3 o la P4 (Mac: QuickTime; Pop!_OS: `Ctrl+Alt+Shift+R`). Los CSV del generador y del monitor ya quedan en `evidencias/`.

10.4 Resultados (23 de octubre de 2026, máquinas reales)

Hechas desde n1 con `docker kill` (equivale a cortar la corriente del nodo) y `tailscale down` (partición), con un generador a 180 cambios/min:

Prueba	Nodo afectado (rol)	Tiempo sin primario	Suma OK	Faltan	Re-sincronización
P8 switchover	n2 (primario) → n1	3,0 s	✓	0	3 nodos sanos en 14 s
P9 `stop patroni`	n1 (primario)	3,0 s	✓	0	3 s
P3 caída brusca	n1 (primario)	**26,1 s**	✓	0	`pg_rewind`, sano en 45 s
P2 caída brusca	n1 (réplica)	0 s	✓	0	4 s
P4 partición	n1 (primario)	se degradó solo a los 11 s; n3 primario a los 22 s	✓	0	réplica, lag 0
P7 10 min caído	n1 (réplica)	0 s	✓	0	se puso al día con el WAL (600 transacciones atrasadas) en 5 s

Verificación final: 3 nodos con datos idénticos, 1,301 movimientos, suma 1,000,000, 0 faltantes. Las 1,301 transacciones que el generador recibió como confirmadas están todas en la base: **ninguna se perdió**. Hubo 8 reintentos, uno por cada cambio de primario, sin duplicados.

Pendientes (requieren estar frente a n2 o n3):

Prueba	Hora	Primario antes	Nodo afectado	Nuevo primario a las	Tiempo sin primario (s)	Suma OK	Faltan 0	Re-sincronizado a las
P1 apagado normal								
P3 con apagado físico real								
P5 réplica sin red								
P6 `sudo reboot`								

--------|------|----------------|---------------|----------------------|------------------------|---------|----------|----------------------|-------
| P1 | | | | | | | | | | | P2 | | | | | | | | | | | P3 | | | | | | | | | | | P4 | | | | | | | | | | | P5 | | | | | | | | | | | P6 | | | | | | | | | | | P7 | | | | | | | | | | | P8 | | | | | | | | | | | P9 | | | | | | | | | | |

11. Paso 8: preparar la demo

Día anterior - ☐ `git pull` en los 3 - ☐ Equipos cargados; Tailscale con sesión iniciada en los 3 - ☐ Hotspot del celular o router probado con los 3 equipos conectados - ☐ Ensayo completo con el guion del plan §15.3

30 minutos antes 1. Conectar los 3 a la red y probar `tailscale ping` entre todos. 2. `./scripts/iniciar.sh` en los 3 (si ya estaba corriendo, solo muestra el estado). 3. Abrir el panel (`./scripts/panel.sh`) en **dos** equipos, o en uno y verlo desde un celular, porque pueden apagar el que estás proyectando. 4. Abrir el panel de HAProxy `http://<IP>:7000`. 5. Arrancar los 3 generadores. 6. Mac: dejar `caffeinate -dimsu` corriendo en una terminal y **no cerrar la tapa**.

Opcional, para limpiar los datos de los ensayos (con todo sano, desde cualquier equipo):

```
source scripts/env_app.sh
python -c "import os, psycopg; c = psycopg.connect(os.environ['HA_DSN'], autocommit=True);
c.execute('TRUNCATE movimientos'); c.execute('UPDATE cuentas SET saldo = 10000')"
```

---

12. Paso 9: informe

Estructura y lista de evidencias en el **plan §16**. Ya hay material: - Teoría y decisiones: plan §1–§3 y §19 (preguntas del catedrático) - Resultados del laboratorio: plan §21.1 - Pendiente: la tabla de §10.4 de esta guía con los datos reales, capturas y la gráfica de cambios por minuto (plan §16.3)

13. Problemas frecuentes

Problema	Solución rápida
`iniciar.sh` espera quórum de etcd para siempre	Los otros nodos no están arriba o no se ven: `tailscale ping`, revisar las IP del `.env`
`password authentication failed`	Las contraseñas no son iguales en los 3 `.env`: corregir y luego `./scripts/pctl.sh reinit hadb <nodo> --force`
Un nodo queda en `start failed` tras volver	`./scripts/pctl.sh reinit hadb <nodo> --force`
etcd: `cluster ID mismatch`	En los 3: `docker compose down` y `docker volume rm hadb_etcd-data`, luego `./scripts/iniciar.sh` (plan §17)
Cambiaron las IP (modo LAN)	Plan §6.4
La Mac no recibe conexiones	Firewall de macOS: permitir Docker (plan §5.1 y §17)
Mac: puertos 5000 y 7000 ocupados (`ControlCe`)	Es el Receptor AirPlay: *Ajustes → General → AirDrop y Handoff → Receptor AirPlay* apagado
Monitor < 90 cambios/min	Revisar que los generadores sigan vivos; si la red es lenta, `--tps 2`
Docker no arranca tras reiniciar (Pop!_OS)	`sudo systemctl enable --now docker`
La Mac se durmió	Despertarla; vuelve sola como réplica

14. Limpieza

- **Laboratorio en la Mac** (cuando ya no se use para ensayar): `bash ./laboratorio/lab.sh destruir`
- **Detener el clúster real** en un equipo, sin perder datos: `docker compose stop`
- **Volver a levantarlo:** `./scripts/iniciar.sh`
- **Borrar todo y empezar desde cero** (en los 3): `docker compose down -v`. **Borra los datos.**